

3. Creación de una Petición AJAX paso a paso

Cuando usamos AJAX, realizamos básicamente **cuatro pasos**:

1. Crear el objeto de petición.
2. Abrir/configurar la petición.
3. Enviar la petición con los datos (si aplica).
4. Procesar la respuesta cuando llegue.

3.1 Petición GET

GET se usa cuando queremos **solicitar información del servidor**, sin modificar nada en él.

Ejemplo práctico: Obtener un archivo JSON con información de productos

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>AJAX GET</title>
</head>
<body>
  <h1>Lista de productos</h1>
  <button id="btnCargar">Cargar productos</button>
  <ul id="listaProductos"></ul>

  <script src="script.js"></script>
</body>
</html>
```

JavaScript (**script.js**)

```
document.getElementById("btnCargar").addEventListener("click",
function() {
  let xhr = new XMLHttpRequest();
  xhr.open("GET", "productos.json", true);

  xhr.onload = function() {
    if (xhr.status === 200) {
      let productos = JSON.parse(xhr.responseText);
      let lista = document.getElementById("listaProductos");
      lista.innerHTML = "";
      productos.forEach(prod => {
        let li = document.createElement("li");
        li.textContent = `${prod.nombre} - ${prod.precio}`;
        lista.appendChild(li);
      });
    } else {
      alert("Error al cargar productos");
    }
  };
});
```

```
        xhr.send();
    });
```

Archivo `productos.json`

```
[
  {"nombre": "Laptop", "precio": 1200},
  {"nombre": "Mouse", "precio": 25},
  {"nombre": "Teclado", "precio": 45}
]
```

Funcionamiento:

- Al hacer clic, se solicita el archivo `productos.json`.
- Los productos se muestran en una lista `<ul>` sin recargar la página.

3.2 Petición POST

POST se usa cuando queremos **enviar información al servidor** (formularios, datos, etc.).

Ejemplo práctico: Enviar datos de un formulario

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>AJAX POST</title>
</head>
<body>
  <h1>Formulario de contacto</h1>
  <form id="formContacto">
    Nombre: <input type="text" name="nombre"><br><br>
    Email: <input type="email" name="email"><br><br>
    <button type="submit">Enviar</button>
  </form>
  <div id="respuesta"></div>

  <script src="script.js"></script>
</body>
</html>
```

JavaScript (script.js)

```
document.getElementById("formContacto").addEventListener("submit",
function(e) {
    e.preventDefault(); // Evita recargar la página

    let xhr = new XMLHttpRequest();
    xhr.open("POST", "procesar_form.php", true);
    xhr.setRequestHeader("Content-Type", "application/x-www-form-
urlencoded");

    xhr.onload = function() {
        if (xhr.status === 200) {
            document.getElementById("respuesta").innerText =
xhr.responseText;
        } else {
            document.getElementById("respuesta").innerText = "Error al
enviar el formulario";
        }
    };

    // Recoger datos del formulario
    let formData = new
FormData(document.getElementById("formContacto"));
    let datos = new URLSearchParams(formData).toString();

    xhr.send(datos);
});
```

Archivo procesar_form.php (simulado en servidor)

```
<?php
$nombre = $_POST['nombre'];
$email = $_POST['email'];
echo "¡Formulario recibido! Nombre: $nombre, Email: $email";
?>
```

Funcionamiento:

- Se evita recargar la página con `e.preventDefault()`.
- Los datos se envían al servidor con `POST`.
- La respuesta se muestra dinámicamente.

3.3 Procesamiento de la respuesta

- Con `XMLHttpRequest`:
 - `xhr.responseText` → devuelve texto plano o JSON.
 - `xhr.responseXML` → devuelve XML.
 - Con `Fetch API`:
 - `response.text()` → texto
 - `response.json()` → JSON
 - `response.blob()` → archivos binarios
-

3.4 Envío de parámetros en formato JSON

Si queremos enviar un objeto JSON:

```
let xhr = new XMLHttpRequest();
xhr.open("POST", "procesar_json.php", true);
xhr.setRequestHeader("Content-Type", "application/json; charset=UTF-8");

xhr.onload = function() {
    console.log(xhr.responseText);
};

let datos = {
    nombre: "Juan",
    edad: 30
};

xhr.send(JSON.stringify(datos));
```

En el servidor (PHP):

```
$input = json_decode(file_get_contents('php://input'), true);
echo "Nombre: " . $input['nombre'] . ", Edad: " . $input['edad'];
```

Ventaja: JSON es más fácil de manejar y enviar objetos complejos.

3.5 Ejercicios propuestos

1. Crear un formulario que envíe dos números al servidor y devuelva la suma usando POST y AJAX.
2. Modificar el ejemplo de productos (GET) para que acepte un parámetro `categoria` y filtre los productos.
3. Enviar un objeto JSON con varios productos y mostrar la respuesta del servidor que indique cuántos productos se recibieron.